

Implementation drills for SD/infra algorithms that get *coded* in rounds (LLD or "write the core of your design"). Per question: why → how → **v1** (interview primary, Python + Java, pros/cons) → **v2 / Follow-up** (variation for the extra requirement, pros/cons). Kept short — full designs live in `sd##` files.

Index

1. [LRU Cache](#) — map + DLL; v2: TTL expiration (报告追问)
 2. [Rate Limiter](#) — token bucket; v2: sliding-window log
 3. [KV Cache](#) — TTL KV store; v2: time-versioned get (981 形)
 4. [Kafka-like Distributed Log](#) — append log + offsets; v2: partitions + consumer groups
-

1. LRU Cache (map + doubly linked list)

Why: bounded memory needs an eviction policy; LRU approximates "keep what's hot" with $O(1)$ ops — the default cache policy everywhere (page cache, Redis `allkeys-lru`, CDN). How: hash map key→node for $O(1)$ lookup; doubly linked list keeps recency order (head = hottest). get/put move the node to head; over capacity → evict tail.

```

class Node:
    __slots__ = ('key', 'val', 'prev', 'next')
    def __init__(self, key=0, val=0):
        self.key, self.val = key, val
        self.prev = self.next = None

class LRUCache:
    def __init__(self, cap):
        self.cap, self.map = cap, {}
        self.head, self.tail = Node(), Node()      # sentinels
        self.head.next, self.tail.prev = self.tail, self.head

    def _unlink(self, n):
        n.prev.next, n.next.prev = n.next, n.prev

    def _front(self, n):
        n.next, n.prev = self.head.next, self.head
        self.head.next.prev = n
        self.head.next = n

    def get(self, key):
        n = self.map.get(key)
        if not n:
            return -1
        self._unlink(n); self._front(n)
        return n.val

    def put(self, key, val):
        if key in self.map:
            n = self.map[key]
            n.val = val
            self._unlink(n); self._front(n)
            return
        if len(self.map) == self.cap:
            lru = self.tail.prev
            self._unlink(lru)
            del self.map[lru.key]
        n = Node(key, val)
        self.map[key] = n
        self._front(n)

# Time: O(1) get/put, Space: O(cap)

```

或 OrderedDict 速写版 (先说它存在再手写 DLL — 面试官多半要手写):

```

from collections import OrderedDict

class LRUCacheOD:
    def __init__(self, cap):
        self.cap = cap
        self.od = OrderedDict()

    def get(self, key):
        if key not in self.od:
            return -1
        self.od.move_to_end(key)      # 最近使用 -> 尾部
        return self.od[key]

    def put(self, key, val):
        if key in self.od:
            self.od.move_to_end(key)
        self.od[key] = val
        if len(self.od) > self.cap:
            self.od.popitem(last=False)  # 弹出最久未用 (头部)

# Time: O(1) get/put – OrderedDict 内部就是 map+DLL, Python 版 LinkedHashMap

```

```

class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int cap;
    LRUCache(int cap) { super(cap, 0.75f, true); this.cap = cap; } // accessOrder=true
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) {
        return size() > cap;
    }
}

// Time: O(1) get/put, Space: O(cap) – say "LinkedHashMap is a map + DLL internally";
// hand-rolled Java DLL version: sd03

```

Pros/cons (v1): O(1) everything, exact LRU; cons — DLL pointer surgery is bug-prone live (sentinels remove null checks), per-entry pointer overhead, not thread-safe (coarse lock or striped locks → sd03).

v2 / Follow-up — TTL expiration (报告原追问: 过期条目读起来像不存在):

```

import time

class LRUCacheTTL(LRUCache):
    def __init__(self, cap):
        super().__init__(cap)
        self.exp = {}                                # key -> expires_at

    def put(self, key, val, ttl=None):
        super().put(key, val)
        if ttl:
            self.exp[key] = time.time() + ttl
        else:
            self.exp.pop(key, None)

    def get(self, key):
        if key in self.exp and time.time() >= self.exp[key]:
            n = self.map.pop(key)                    # lazy eviction
            self._unlink(n)
            del self.exp[key]
            return -1
        return super().get(key)

# Time: O(1); expired entries linger until touched (capacity still counts them)

```

```

class TtlLru<K, V> {
    private final LRUCache<K, Entry<V>> lru;
    record Entry<V>(V val, long expiresAt) {}
    TtlLru(int cap) { this.lru = new LRUCache<>(cap); }

    V get(K key) {
        Entry<V> e = lru.get(key);
        if (e == null) return null;
        if (e.expiresAt() > 0 && System.currentTimeMillis() >= e.expiresAt()) {
            lru.remove(key);                        // lazy expiry
            return null;
        }
        return e.val();
    }

    void put(K key, V val, long ttlMs) {
        lru.put(key, new Entry<>(val, ttlMs > 0 ? System.currentTimeMillis() + ttlMs : 0));
    }
}

// wrap-don't-modify: TTL lives in the value record, LRU stays untouched

```

Pros/cons (v2): lazy expiry keeps O(1) and zero background work; cons — dead entries hold capacity until touched (follow-up answer: background sweeper or expiry min-heap, O(log n) — Redis does lazy + sampled active). 说法: "先 lazy, 讲清代价, 再给 active 选项".

2. Rate Limiter

Why: protect a service from bursts/abuse and enforce per-tenant fairness; every gateway has one (sd01 full design). How (token bucket): bucket holds up to `cap` tokens, refilled at `rate`/s; a request spends 1 token; empty bucket → reject. Allows bursts up to `cap` while bounding average rate. Lazy refill on access — no timer thread.

```
import time

class TokenBucket:
    def __init__(self, rate, cap):
        self.rate, self.cap = rate, cap
        self.tokens = cap
        self.ts = time.monotonic()

    def allow(self, n=1):
        now = time.monotonic()
        self.tokens = min(self.cap, self.tokens + (now - self.ts) * self.rate)
        self.ts = now
        if self.tokens >= n:
            self.tokens -= n
            return True
        return False

# Time: O(1), Space: O(1) per key – 挂在 dict[key] 上即 per-client
```

```
class TokenBucket {
    private final double rate; private final int cap;
    private double tokens; private long ts = System.nanoTime();
    TokenBucket(double rate, int cap) { this.rate = rate; this.cap = cap; this.tokens = cap; }

    synchronized boolean allow() {
        long now = System.nanoTime();
        tokens = Math.min(cap, tokens + (now - ts) / 1e9 * rate);
        ts = now;
        if (tokens >= 1) { tokens -= 1; return true; }
        return false;
    }
}

// synchronized: state update must be atomic; per-key buckets in ConcurrentHashMap
```

Pros/cons (v1): O(1) memory, burst-friendly, industry default; cons — "3 in any rolling second" 这类精确窗口语义它不保证 (平均速率 ≠ 严格窗口), needs per-key map cleanup.

v2 / Follow-up — sliding-window log (严格 "任意窗口内 ≤ N" 语义, 面试常用来对比):

```

import time
from collections import deque

class SlidingWindowLog:
    def __init__(self, limit, window_s):
        self.limit, self.win = limit, window_s
        self.log = deque()                # request timestamps

    def allow(self):
        now = time.monotonic()
        while self.log and self.log[0] <= now - self.win:
            self.log.popleft()
        if len(self.log) < self.limit:
            self.log.append(now)
            return True
        return False

# Time: amortized O(1), Space: O(limit) per key

```

```

class SlidingWindowLog {
    private final int limit; private final long winNanos;
    private final ArrayDeque<Long> log = new ArrayDeque<>();
    SlidingWindowLog(int limit, long winMs) { this.limit = limit; this.winNanos = winMs * 1_000_000; }

    synchronized boolean allow() {
        long now = System.nanoTime();
        while (!log.isEmpty() && log.peekFirst() <= now - winNanos) log.pollFirst();
        if (log.size() < limit) { log.addLast(now); return true; }
        return false;
    }
}

// exact rolling-window semantics; memory grows with limit

```

Pros/cons (v2): 精确窗口、无边界突刺; cons — $O(\text{limit})$ 内存 per key (高限额贵), 分布式版要 Redis ZSET/Lua (sd01)。对比一句话: bucket = 平均速率+允许突发省内存; log = 严格窗口费内存; fixed-window counter = 最便宜但边界双倍突刺。计数变体 (dropped-requests, 丢弃也计数) 见 [../1.coding/0.1p3a.md#orc-rate-limiter](#)。

3. KV Cache (TTL KV store)

Why: 几乎所有 service 的第一层状态 — session、config、feature flag; Redis-lite 的核心就是 dict + 过期; 报告里的 Redis-like store / "design a cache" 都落在这。How: dict 存值 + dict 存过期时间; get 时 lazy 判断过期; 再加一个 min-heap 按到期时间做 active sweep (可选)。

```

import time, heapq

class KVCache:
    def __init__(self):
        self.data = {}
        self.exp = {}                # key -> expires_at
        self.heap = []               # (expires_at, key) for sweep

    def put(self, key, val, ttl=None):
        self.data[key] = val
        if ttl:
            at = time.time() + ttl
            self.exp[key] = at
            heapq.heappush(self.heap, (at, key))
        else:
            self.exp.pop(key, None)

    def get(self, key):
        at = self.exp.get(key)
        if at and time.time() >= at:    # lazy expiry
            self.delete(key)
            return None
        return self.data.get(key)

    def delete(self, key):
        self.data.pop(key, None)
        self.exp.pop(key, None)

    def sweep(self, limit=100):        # active expiry, call periodically
        now = time.time()
        while self.heap and self.heap[0][0] <= now and limit:
            at, key = heapq.heappop(self.heap)
            if self.exp.get(key) == at:    # ignore stale heap entries
                self.delete(key)
            limit -= 1

# Time: get/put O(1) (+O(log n) heap push), Space: O(n)

```

```

class KVCache<K, V> {
    record Entry<V>(V val, long expiresAt) {}
    private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();

    void put(K key, V val, long ttlMs) {
        long at = ttlMs > 0 ? System.currentTimeMillis() + ttlMs : 0;
        map.put(key, new Entry<>(val, at));
    }

    V get(K key) {
        Entry<V> e = map.get(key);
        if (e == null) return null;
        if (e.expiresAt() > 0 && System.currentTimeMillis() >= e.expiresAt()) {
            map.remove(key, e); // lazy expiry, CAS-safe
            return null;
        }
        return e.val();
    }
}

// ConcurrentHashMap + remove(key, value) = no lost update on concurrent put

```

Pros/cons (v1): O(1) 全操作、lazy+sweep 组合是 Redis 真实做法; cons — 无淘汰策略 (配 \$1 LRU 成完整 cache), heap 里有 stale entry (put 覆盖时) 需 version 核对.

v2 / Follow-up — time-versioned get (981 形: `get(key, ts)` 返回 $\leq ts$ 的最新版本; 报告电面原题):

```

from bisect import bisect_right

class TimeMap:
    def __init__(self):
        self.d = {} # key -> [(ts, val)] 追加有序

    def set(self, key, val, ts):
        self.d.setdefault(key, []).append((ts, val))

    def get(self, key, ts):
        arr = self.d.get(key, [])
        i = bisect_right(arr, (ts, chr(127))) - 1
        return arr[i][1] if i >= 0 else ""

# Time: set O(1), get O(log n); 前提: 每 key 的 ts 递增 (题面保证)

```



```

class TimeMap {
    private final Map<String, TreeMap<Integer, String>> d = new HashMap<>();

    void set(String key, String val, int ts) {
        d.computeIfAbsent(key, k -> new TreeMap<>()).put(ts, val);
    }

    String get(String key, int ts) {
        TreeMap<Integer, String> m = d.get(key);
        if (m == null) return "";
        Map.Entry<Integer, String> e = m.floorEntry(ts);    // 最大的 <= ts
        return e == null ? "" : e.getValue();
    }
}
// TreeMap.floorEntry = Java 版 bisect; 不要求 ts 递增, 更通用

```

Pros/cons (v2): 版本化读 = 审计/快照语义, list+bisect 最省 (靠递增前提); TreeMap 通用但 $O(\log n)$ set. 追问 "内存无限涨" → 按时间截断老版本 / 压缩 (Kafka compaction 同思想, §4).

4. Kafka-like Distributed Log

Why: 服务解耦的骨干 — 有序、可重放、多消费者的事件流 (sd15 全设计); "写个简化 log" 是 LLD 常客. How: append-only 数组当 segment, offset = 下标; 消费者自己持有 offset 拉取 — replay 免费; 稀疏索引让磁盘版 $O(\log)$ 定位.

```

class Log:                                # 单 partition
    def __init__(self):
        self.records = []                # append-only

    def append(self, record):
        self.records.append(record)
        return len(self.records) - 1      # offset

    def read(self, offset, max_n=100):
        return self.records[offset:offset + max_n]    # 消费者管理自己的 offset
# Time: append  $O(1)$ , read  $O(k)$ ; replay = 从任意 offset 重读

```

```

class Log {
    private final List<byte[]> records = new ArrayList<>();

    synchronized long append(byte[] rec) {
        records.add(rec);
        return records.size() - 1;                // offset
    }

    synchronized List<byte[]> read(long offset, int maxN) {
        int from = (int) offset;
        if (from >= records.size()) return List.of();
        return List.copyOf(records.subList(from, Math.min(from + maxN, records.size())));
    }
}
// 单写锁足够 (append-only); 磁盘版: segment 文件 + 稀疏索引 (sd15 §6)

```

Pros/cons (v1): 极简却承载全部核心语义 (offset、顺序、重放); cons — 单 partition 无并行、无持久化、无保留策略 (说出 segment + retention 的扩展即可).

v2 / Follow-up — partitions + consumer groups (追问: 怎么并行? 多个消费者怎么各读各的?):

```

class Topic:
    def __init__(self, partitions=3):
        self.parts = [Log() for _ in range(partitions)]
        self.offsets = {}                # (group, partition) -> next offset

    def produce(self, key, record):
        p = hash(key) % len(self.parts)    # 同 key 同 partition -> 有序
        return p, self.parts[p].append(record)

    def poll(self, group, partition, max_n=100):
        off = self.offsets.get((group, partition), 0)
        return self.parts[partition].read(off, max_n)    # 处理完再 commit

    def commit(self, group, partition, next_offset):
        self.offsets[(group, partition)] = next_offset    # commit-after = at-least-once
# 每 partition 同组内只给一个消费者 -> partition 数 = 并行上限

```

```

class Topic {
    private final Log[] parts;
    private final ConcurrentHashMap<String, Long> offsets = new ConcurrentHashMap<>();
    Topic(int n) { parts = new Log[n]; Arrays.setAll(parts, i -> new Log()); }

    int produce(String key, byte[] rec) {
        int p = Math.floorMod(key.hashCode(), parts.length);
        parts[p].append(rec);
        return p;
    }
    List<byte[]> poll(String group, int p, int maxN) {
        long off = offsets.getOrDefault(group + "/" + p, 0L);
        return parts[p].read(off, maxN);
    }
    void commit(String group, int p, long nextOff) {
        offsets.put(group + "/" + p, nextOff);          // 崩溃在 commit 前 -> 重投递
    }
}
// at-least-once: 先处理后 commit; 下游按 (partition, offset) 幂等去重

```

Pros/cons (v2): key→partition 保 per-key 有序 + 水平并行, group offset = 多消费者互不干扰; cons — 热 key 偏斜 (加盐), commit 前崩溃有重复 (说 at-least-once + 幂等), 复制/ISR/高水位是下一层 (sd15 §7)。对比一句: queue 按消息 ack 发一次, log 按 offset 可重放多读 — 别拿 SQS 语义答 Kafka 题。